



Centrifuge v3.3 Part 2 Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

LonelySloth

Taridoku

fuzious

0xBeastBoy

August 31, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	4
7	Findings	6
7.1	Medium Risk	6
7.1.1	Non-Atomic Linked Asset/Liability Repricing Can Publish an Incorrect PPS	6
7.1.2	A pool manager can overwrite or preempt another pool's vault registration	7
7.2	Low Risk	8
7.2.1	An equal-price first commit leaves the share-price guard unarmed	8
7.2.2	Non-ERC-7575 Share-Token Vault Unlink Reverts	9
7.2.3	Simple Price Manager Equal Price Skips Initialization and Fee Accrual	10
7.3	Informational	11
7.3.1	Missing specification of permissions for Request Manager	11
7.3.2	Cross-Network NAV Initializer Poisons Accounting Role	11
7.3.3	Failed setPoolAdapters Retry Bypasses Session Revocation	12
7.3.4	IHoldings.decrease documents the wrong valuation behavior	12
7.3.5	Holdings.increase unnecessarily quotes a zero realized value	13
7.3.6	ITokenBridge documents the wrong refund and return behavior	13
7.3.7	Cross-chain documentation names the wrong execution components	13
7.3.8	Spoke performs a redundant share-class registry lookup	14
7.3.9	VaultRouter queries vault.share() twice	14

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The audited scope spans Centrifuge's immutable core and the modular extension layer built on top of it. Core holds the minimal set of contracts needed to register pools, account for holdings, manage share classes, and move state between chains; vaults, managers, hooks, valuations, and adapters plug into it without requiring any change to it.

A pool is one balance sheet kept consolidated across every chain it operates on. Hub runs the control plane on a single chain through Holdings, Accounting, ShareClassManager, and HubRegistry, holding the books but never custodying user assets, while Spoke is the pool's local presence on every chain it touches, deploying share tokens through a pluggable registrar and custodying assets in per-pool, per-share-class escrows. SnapshotQueue nets share deltas per share class and accumulates asset flows gross per asset, flushing one update to the Hub so routine deposits and withdrawals do not each cross a chain boundary.

Manager authority is bounded onchain by the pool's manifest, which Hub and Spoke both enforce on every guarded manager call: in-policy calls run synchronously, while out-of-policy ones must be pre-authorized, mature past a delay, and can be vetoed by the sentinels Supervisor tracks. Extension managers cover the operational surface around it, with OnOffRamp for asset on- and off-ramping, QueueManager for batched syncing, NAVManager and SimplePriceManager for NAV and share-price accounting, and OracleValuation for feeder-supplied asset prices, alongside OnchainPM, a per-pool weiroll engine running Merkle-proof-authorized strategist workflows under SlippageGuard and CircuitBreakerGuard bookends.

AsyncVault and SyncDepositVault expose ERC-4626 and ERC-7540 deposit and redeem flows, settled through AsyncRequestManager and BatchRequestManager over ShareToken, an ERC-20 with ERC-1404 restrictions enforced by transfer hooks. Across chains, MessageDispatcher and MessageProcessor serialize and route payloads that MultiAdapter forwards only once a configurable quorum of the Axelar, Chainlink, Hyperlane, and LayerZero adapters agrees, with StandbyAdapter filling a low-cost quorum slot, AdapterFailover replacing an adapter set that has gone dark, and TokenBridge relaying share transfers from spoke to spoke via the hub.

5 Audit Scope

The scope of the audit is the diff between commit 23b3e9666d60c2d014092e4b60e4360017e23509 and commit cbb4025a2ec6f25edab4610d4f4602d886d4c0b3 for all the files in the:

```
src/
├── adapters/
│   ├── AxelarAdapter.sol
│   ├── ChainlinkAdapter.sol
│   ├── HyperlaneAdapter.sol
│   ├── LayerZeroAdapter.sol
│   └── StandbyAdapter.sol
├── admin/
│   ├── GasService.sol
│   ├── OpsGuardian.sol
│   ├── ProtocolGuardian.sol
│   └── Root.sol
├── bridge/
│   └── TokenBridge.sol
├── core/
│   ├── hub/
│   │   ├── Accounting.sol
│   │   ├── Holdings.sol
│   │   ├── Hub.sol
│   │   ├── HubHandler.sol
│   │   ├── HubRegistry.sol
│   │   └── ShareClassManager.sol
│   ├── libraries/
│   │   └── PricingLib.sol
│   ├── messaging/
│   │   ├── libraries/
│   │   │   └── MessageLib.sol
│   │   ├── MessageDispatcher.sol
│   │   ├── MessageProcessor.sol
│   │   └── MultiAdapter.sol
│   ├── spoke/
│   │   ├── factories/
│   │   │   └── PoolEscrowFactory.sol
│   │   ├── types/
│   │   │   └── Price.sol
│   │   ├── SnapshotQueue.sol
│   │   ├── Spoke.sol
│   │   ├── SpokeHandler.sol
│   │   └── SpokeRegistry.sol
│   ├── types/
│   │   ├── PoolId.sol
│   │   └── RequestId.sol
│   └── utils/
│       ├── ContractUpdateLib.sol
│       ├── ContractUpdater.sol
│       └── ContractUpdaterForwarder.sol
├── hooks/
│   ├── accounting/
│   │   ├── NAVManager.sol
│   │   └── SimplePriceManager.sol
│   └── bridge/
│       └── BridgeCircuitBreaker.sol
├── managers/
│   ├── adapters/
│   │   └── AdapterFailover.sol
│   ├── hub/
│   │   └── Supervisor.sol
```

```

├── spoke/
│   ├── guards/
│   │   ├── CircuitBreakerGuard.sol
│   │   └── SlippageGuard.sol
│   ├── AccountingToken.sol
│   ├── OnOffRamp.sol
│   └── QueueManager.sol
├── manifests/
│   └── hub/
│       └── StdHubManifest.sol
├── misc/
│   ├── libraries/
│   │   ├── ExcessivelySafeCallLib.sol
│   │   └── SafeTransferLib.sol
│   ├── types/
│   │   └── D18.sol
│   ├── Escrow.sol
│   └── Recoverable.sol
├── token/
│   ├── hooks/
│   │   ├── libraries/
│   │   │   └── UpdateRestrictionMessageLib.sol
│   │   ├── BaseTransferHook.sol
│   │   ├── FreelyTransferable.sol
│   │   ├── FreezeOnly.sol
│   │   ├── FullRestrictions.sol
│   │   └── RedemptionRestrictions.sol
│   ├── ShareToken.sol
│   └── ShareTokenRegistrar.sol
├── utils/
│   └── SubsidyManager.sol
├── valuations/
│   └── OracleValuation.sol
├── vaults/
│   ├── factories/
│   │   ├── AsyncVaultFactory.sol
│   │   └── SyncDepositVaultFactory.sol
│   ├── AsyncRequestManager.sol
│   ├── AsyncVault.sol
│   ├── BaseVaults.sol
│   ├── BatchRequestManager.sol
│   ├── SyncDepositVault.sol
│   ├── SyncManager.sol
│   └── VaultRouter.sol
└── src-ir/
    └── OnchainPM.sol

```

6 Executive Summary

Over the course of 5 days, the Burra Security team conducted an audit on the [Centrifuge v3.3 Part 2](#) smart contracts provided by [Centrifuge](#). In this period, a total of 14 issues were found.

Summary

Project Name	Centrifuge v3.3 Part 2
Repository	protocol-internal
Commit	cbb4025a2ec6...
Audit Timeline	Jul 28th - Aug 3rd
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	3
Informational	9
Gas Optimizations	0
Total Issues	14

Summary of Findings

[M-1] Non-Atomic Linked Asset/Liability Repricing Can Publish an Incorrect PPS	Acknowledged
[M-2] A pool manager can overwrite or preempt another pool's vault registration	Resolved
[L-1] An equal-price first commit leaves the share-price guard unarmed	Resolved
[L-2] Non-ERC-7575 Share-Token Vault Unlink Reverts	Resolved
[L-3] Simple Price Manager Equal Price Skips Initialization and Fee Accrual	Resolved
[I-1] Missing specification of permissions for Request Manager	Resolved
[I-2] Cross-Network NAV Initializer Poisons Accounting Role	Resolved
[I-3] Failed setPoolAdapters Retry Bypasses Session Revocation	Acknowledged
[I-4] IHoldings.decrease documents the wrong valuation behavior	Resolved
[I-5] Holdings.increase unnecessarily quotes a zero realized value	Resolved
[I-6] ITokenBridge documents the wrong refund and return behavior	Resolved
[I-7] Cross-chain documentation names the wrong execution components	Resolved
[I-8] Spoke performs a redundant share-class registry lookup	Resolved
[I-9] VaultRouter queries vault.share() twice	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Non-Atomic Linked Asset/Liability Repricing Can Publish an Incorrect PPS

Target: `src/managers/spoke/OnOffRamp.sol`, `src/managers/spoke/AccountingToken.sol`, `src/valuations/OracleValuation.sol`, `src/core/hub/Hub.sol`, `src/core/hub/Holdings.sol`, `src/hooks/accounting/NAVManager.sol`, `src/hooks/accounting/SimplePriceManager.sol`, `src/manifests/hub/StdHubManifest.sol`, and `src/vaults/SyncManager.sol`

Severity:

- Impact: High
- Likelihood: Low

Description: `OnOffRamp.deposit()` records equal raw amounts of a real asset and its liability `AccountingToken`. These legs should cancel in NAV when valued at the same price. Although the liability token ID encodes the underlying asset and mirrors its decimals, the Hub represents each leg with a distinct `AssetId`, and `OracleValuation` stores their prices independently without binding the liability price to the underlying asset price.

Each `setPrice()` or `fromSpoke()` call updates only one `AssetId` and immediately calls `Hub.updateHoldingValue()`. That call revalues the selected holding, journals the change, and runs the network's snapshot hook. Therefore, when the network has a valid snapshot without a deficit and uses `NAVManager` with `SimplePriceManager`, the first leg's update can recompute NAV and commit a new pool-per-share price before the matching leg is repriced.

Before v3.3, on-chain accounting restricted `updateHoldingValue()` to `NAVManager`, so the callback from `OracleValuation` reverted. V3.3 adds an earlier fast path for the configured `OracleValuation`; once it is also authorized as a Hub manager, singleton oracle revaluation succeeds without enforcing linked-price atomicity.

For example, a pool with NAV and issuance of 1,000 and an offsetting asset/liability pair worth 100 per leg has a share price of 1. If both leg prices move from 1 to 2, the final NAV remains 1,000; however, updating the asset first temporarily publishes a share price of 1.1, while updating the liability first publishes 0.9. An honest feeder can create this state through consecutive transactions even when both submitted prices are correct.

The manifest's rate and absolute-delta checks can bound individual share-price moves but do not enforce linked-leg consistency, so an intermediate move within those bounds remains valid.

Crucially, this impact does not require cross-chain price propagation or message ordering. When the affected Spoke uses the Hub deployment's `localCentrifugeId`, a manager or keeper can call `Hub.notifySharePrice()` during the interval between the two oracle updates. `MessageDispatcher.sendNotifyPricePoolPerShare()` then takes its synchronous local branch and calls `SpokeHandler.updatePricePoolPerShare()` directly; no gateway message is created, delayed, or reordered. If the linked synchronous vault has reserve capacity and `SyncManager` has no separate valuation override, `deposit()` reads this intermediate registry PPS and immediately issues shares at it. A low transient PPS therefore overmints shares and dilutes existing holders, while a high transient PPS underallocates shares to the depositor. Correcting the second oracle leg does not unwind shares already issued. The loss is limited to the affected pool and the configured `maxReserve`, but can still be material.

Impact is High because a completed conversion at a false PPS transfers economic value through irreversible share issuance. Likelihood is Low because the path requires `NAVManager/SimplePriceManager`, a valid non-deficit snapshot, two separately submitted linked-price updates, an authorized local price notification in the intervening window, and an eligible synchronous deposit before correction. High impact × Low likelihood yields Medium, and the explicit `NAVManager/SimplePriceManager` cap independently limits the final severity to Medium.

Recommended Mitigation: Make economically linked repricing atomic with respect to NAV publication. Prefer deriving an `AccountingToken` liability's valuation from its encoded underlying asset price. If separate price records are required, bind the two `AssetId` values and stage their updates in a batch or epoch; reject incomplete or mismatched epochs, then revalue all linked holdings before invoking the snapshot hook or publishing a share price. Do not call `updateHoldingValue()` or the snapshot hook after each batch member. Add tests covering either update order and assert that neither the Hub nor Spoke exposes an intermediate pool-per-share price and that synchronous deposits cannot use one.

Centrifuge: Acknowledged. Batching can be ensured through proper gas configuration. The price circuit breaker in the hub policy provides additional protections. We will implement batched price updates in a future upgrade.

BurraSec: Acknowledged

7.1.2 A pool manager can overwrite or preempt another pool's vault registration

Target: SpokeHandler.sol, SpokeRegistry.sol, SyncManager.sol, BaseVaults.sol

Severity:

- Impact: Medium
- Likelihood: Medium

Description: SpokeRegistry.registerVault rejects only currently linked addresses; it neither rejects previously registered addresses nor requires the returned address to contain code. After Pool A unlinks vault V, a Pool B manager can use a custom factory that returns V. Because V is currently unlinked, its Pool A registration is overwritten and it is linked to Pool B, preventing Pool A from normally relinking it. Also, Pool B can pre-register and link the deterministic address of Pool A's future vault before code is deployed there. Pool A's later deployment then reverts during registration because the address is already linked. Neither attack depends on reading or validating the vault's own pool or share-class IDs. They both let a manager trusted only for Pool B capture Pool A's global vault address.

Recommended Mitigation: Require returned vault addresses to contain code and to have never been registered. Any reassignment should require an explicit governance operation authorized against the existing record.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: Fix looks good

7.2 Low Risk

7.2.1 An equal-price first commit leaves the share-price guard unarmed

Target: `src/policies/hub/StdHubPolicy.sol`, `src/hooks/accounting/SimplePriceManager.sol`, `src/core/hub/Hub.sol`, `src/core/hub/ShareClassManager.sol`

Severity:

- Impact: Medium
- Likelihood: Low

Description: PR 309 fixed equal-price updates being skipped by making `SimplePriceManager.onUpdate()` commit every recomputed price. It also changed `StdHubPolicy.enforce()` to advance `lastPriceUpdate` only when the incoming raw price differs from the price currently stored in `ShareClassManager`. Because policy enforcement runs before the Hub stores the new price, this comparison fails to arm the policy whenever its first successful update has the same raw price as the stored value.

For a fresh share class with nonzero issuance and zero NAV, `SimplePriceManager` computes zero PPS. `_checkSharePrice()` admits the first update because `lastPriceUpdate == 0`, while `enforce()` compares the incoming zero with the pre-write default zero and leaves the baseline unset. The Hub then commits (`price = 0`, `computedAt = block.timestamp`).

The same state mismatch recurs when a pool installs a fresh `StdHubPolicy` instance. The new policy's `lastPriceUpdate` mapping starts at zero, while the committed price and nonzero `computedAt` survive in `ShareClassManager`. Because `SimplePriceManager` now commits every recomputation, the first update under the replacement policy can be a no-op recommit. `_checkSharePrice()` admits it as the first update, but `enforce()` sees `newPrice == lastPrice` and again leaves the policy baseline unset.

In either case, the next real price move is still treated as the first update and bypasses the same-block restriction, rate limit, absolute-delta cap, and authorization and sentinel-veto flow. That move finally initializes `lastPriceUpdate`. The bypass therefore occurs once per fresh policy state and share class and can reopen after replacement with another fresh guarded policy instance. The directly demonstrated impact is the one-time bypass of the configured price-control boundary; producing an incorrect PPS or financial loss requires a separate accounting-input manipulation path.

This is a regression introduced by the fix for the earlier equal-price issue, not a residual instance of it: the Hub now commits the valid zero price, but the independently introduced policy comparison fails to record that commit as the executed baseline.

Recommended Mitigation: Use the policy's own state to detect an uninitialized baseline, so the first successful update under that policy arms the guard even when it recommits the stored raw price:

```
(D18 lastPrice,) = shareClassManager.pricePoolPerShare(poolId, scId);
if (lastPriceUpdate[poolId][scId] == 0 || newPrice.raw() != lastPrice.raw()) {
    lastPriceUpdate[poolId][scId] = uint64(block.timestamp);
}
```

This preserves the no-op protection after initialization: later equal-price recomputations leave an established baseline unchanged.

Add regression tests that (1) commit a zero PPS and assert both `computedAt != 0` and `lastPriceUpdate != 0`, and (2) install a fresh policy over an already committed nonzero PPS, execute a no-op recommit, and assert that the new policy's `lastPriceUpdate` is initialized. In both cases, verify that a subsequent same-block or cap-exceeding real move requires a matured authorization.

Centrifuge: Fixed in [centrifuge/protocol-internal#352](#)

BurraSec: Fix looks good

7.2.2 Non-ERC-7575 Share-Token Vault Unlink Reverts

Target: `src/core/spoke/SpokeRegistry.sol` (`linkVault`, `unlinkVault`, and `_pointVault`), `src/core/spoke/interfaces/IRegistrar.sol` (`updateVault`), `src/vaults/AsyncVault.sol` (`requestDeposit`), `src/vaults/AsyncRequestManager.sol` (request handling, fulfillment callbacks, and `_requestVault`), and `src/vaults/SyncManager.sol` (`_maxDeposit`)

Severity:

- Impact: Medium
- Likelihood: Low

Description: `IRegistrar.updateVault` explicitly permits registrars for share-token standards without an ERC-7575 vault pointer to no-op. `SpokeRegistry.linkVault` supports this configuration because it marks the vault as linked and delegates the pointer update to the registrar. However, `unlinkVault` first clears `isLinked` and then, for ERC-20 assets, `_pointVault` directly calls `IERC7575Share(token).vault(asset)` before asking the registrar to clear the pointer. If the otherwise compatible share token does not implement that getter, the call reverts or its empty return data fails decoding, rolling back the entire transaction and leaving the vault linked. Consequently, the linked-bit gates in the synchronous and asynchronous managers continue to admit activity for a vault governance cannot deactivate through the normal unlink flow.

The same abstraction violation can strand accepted asynchronous requests. `AsyncVault.requestDeposit` records the request through `AsyncRequestManager` and then transfers the user's assets into the pool escrow without reading the optional pointer. Aggregate approval and share-issuance callbacks also succeed without it. In contrast, each per-user fulfillment callback calls `_requestVault`, which directly resolves the vault through `shareToken.vault(asset)`. For a pointer-less token, the inbound per-user callback reverts before updating user state whether it carries fulfilled amounts or cancellation refunds. Sending the cancellation request may succeed, but its response cannot make the refund claimable, and retrying the callback repeats the failure. The analogous redeem path is affected only when the registrar also supports the privileged share transfer required to escrow shares. Once this permitted pointer-less configuration is linked, every per-user fulfillment or cancellation callback for the affected (`poolId`, `shareClassId`, `assetId`) tuple deterministically hits the missing getter. Users' deposited assets—or, under that redeem prerequisite, redemption shares—can remain locked because the ordinary fulfill, cancel, retry, and unlink flows cannot recover them; governance must introduce a compatible routing or recovery path.

Recommended Mitigation: Keep token-standard-specific pointer handling behind the registrar. Add a conditional-clear registrar operation that receives the expected current vault, allowing ERC-7575 registrars to clear only a matching pointer and pointer-less registrars to no-op without preventing `isLinked` from being cleared. For asynchronous requests, pin the vault address when the request is created or resolve it from registry-owned routing state rather than the share token. If pointer-less tokens are not intended to support async vaults, validate the required capability before linking and reject requests before transferring user funds. Add regression tests proving that pointer-less tokens can link and unlink; deposits and, when privileged share transfer is supported, redemptions can be fulfilled or cancelled, or are rejected before funds enter escrow; and an ERC-7575 pointer is cleared only when it still equals the expected vault.

Centrifuge: Fixed the unlinking issue in [centrifuge/protocol-internal#307](#) and confirmed that non-ERC7575 tokens will be supported in a future vault version; meanwhile, the limitation can be managed by the pool manager.

BurraSec: Acknowledged the fix and accepted the remaining non-ERC7575 async limitation as a known gap pending future support.

7.2.3 Simple Price Manager Equal Price Skips Initialization and Fee Accrual

Target: `src/hooks/accounting/SimplePriceManager.sol` (`onUpdate`), `src/core/hub/Hub.sol` (`updateSharePrice` and `notifySharePrice`), `src/core/hub/ShareClassManager.sol` (`updateSharePrice`), `src/core/spoke/SpokeRegistry.sol` (`pricePoolPerShare`), `src/core/spoke/types/Price.sol` (`isValid`), and `src/vaults/SyncManager.sol` (`pricePoolPerShare`)

Severity:

- Impact: Low
- Likelihood: Medium

Description: `SimplePriceManager.onUpdate()` updates its NAV and issuance metrics, calculates the new pool-per-share price, and compares it only with the raw price stored in `ShareClassManager`. If the values are equal, it returns without considering the stored `computedAt` timestamp. This bypasses `Hub.updateSharePrice()`, including the timestamp write, configured fee-accrual callback, and normal update events.

One edge case occurs when a share class has nonzero issuance but zero aggregate NAV. Its calculated price is zero, which equals the default uninitialized (`price, computedAt`) = (0, 0) state. The early return therefore leaves `computedAt` unset. If this value is notified to a Spoke, `Price.isValid()` rejects it, and a `SyncManager` relying on the registry price reverts instead of observing a valid zero price. This requires the narrow condition of outstanding shares with aggregate NAV reduced to zero, affects an already-zero-value share class, and can be recovered through a later nonzero price or an authorized configuration/update path. Accordingly, this initialization subcase has low impact and low likelihood.

The same shortcut is more likely for initialized share classes because proportional NAV and issuance changes, or fixed-point rounding, can leave the calculated price unchanged between accounting snapshots. Each such snapshot skips the configured `IFeeAccrual accrue()` callback and the update events. However, the repository defines only the fee-accrual interface and does not include a production implementation proving permanent fee loss. The demonstrated consequence is therefore limited to missed callback timing or delayed processing until another accrual-triggering action.

Recommended Mitigation: Do not use raw price equality as a complete no-op. At minimum, call `Hub.updateSharePrice()` when the stored `computedAt` is zero so that a legitimate initial zero price receives a valid timestamp. For initialized equal-price snapshots, either route the snapshot through the normal Hub update path or provide an explicit, equivalently authorized callback path so configured fee processing and update observability are preserved. Add regression tests for nonzero issuance with zero NAV and for an initialized equal-price snapshot with a configured fee-accrual callback.

Centrifuge: Fixed in [centrifuge/protocol-internal#309](#)

BurraSec: The fix looks good

7.3 Informational

7.3.1 Missing specification of permissions for Request Manager

Target: Documentation.

Severity: Informational

Description: There are multiple privileged roles within pools, which are mostly documented. However the role of "Request Manager" isn't properly specified.

Recommended Mitigation: Add "Request Manager" to the "Pool Access Levels" documentation at <https://docs.centrifuge.io/developer/security/pool-access-levels/>

Centrifuge: Fixed at: <https://docs.centrifuge.io/developer/security/pool-access-levels/#request-manager>

BurraSec: Documentation fixed as recommended.

7.3.2 Cross-Network NAV_INITIALIZER Poisons Accounting Role

Target: `src/hooks/accounting/NAVManager.sol`, `src/hooks/accounting/interfaces/INAVManager.sol`, `src/core/spoke/Spoke.sol`, `src/core/messaging/MessageProcessor.sol`, `src/core/utils/Envoy.sol`, `src/core/hub/Hub.sol`, `src/core/hub/Holdings.sol`, `src/manifests/hub/StdHubManifest.sol`, `src/managers/spoke/OnOffRamp.sol`, `src/hooks/accounting/SimplePriceManager.sol`

Severity: Informational

Description: `Spoke.managerCall()` lets any caller send a manager payload while preserving the authenticated source (`centrifugeId`, `sender`) through `MessageProcessor` and `Envoy`. `NAVManager.fromSpoke()` verifies that tuple against `manager[poolId][centrifugeId][sender]`, but does not require the payload's `assetId.centrifugeId()` to match the authenticated network. The subsequent holding and liability initialization paths instead derive all accounting accounts and the initialization check from the payload-selected network.

Once the pool's default valuation and target network B are initialized, a manager authorized only for network A can submit an `InitializeHolding` or `InitializeLiability` payload for an uninitialized, registered network-B asset and valid share class. Under on-chain accounting, `StdHubManifest` permits `NAVManager` to call `Hub.initializeHolding()` immediately; `Hub` validates the share class, asset registration, and account existence, but receives no authenticated source network to compare against the asset. Each newly registered asset therefore has a first-initialization window. A later-added share class can also expose a one-time opposite-role window for an existing asset while that role's distinct per-asset account remains uncreated. A message that initially fails because a prerequisite is missing can be permissionlessly retried after that prerequisite is configured while the sender remains authorized.

The network-A manager can consequently assign a network-B asset the wrong accounting role. `Holdings.initialize()` uses the valuation address as its initialization sentinel and rejects later initialization of the same (`poolId`, `shareClassId`, `assetId`), so the legitimate network-B initializer cannot correct the classification through the normal flow. Subsequent amount and valuation updates are journaled to the attacker-selected account layout: a liability initialized as a holding contributes through equity and gain/loss accounts, while a real asset initialized as a liability contributes through expense and liability accounts. For example, poisoning one leg of an `OnOffRamp` deposit breaks the intended offset between the deposited asset and its accounting-token liability. The resulting network NAV is propagated by `SimplePriceManager` into the pool's price per share, which can inflate or suppress subsequent issuance and redemption pricing. Exposure remains limited to the affected pool and misclassified holdings, while recovery requires privileged account-slot rewiring or another governance action. Pool managers are fully trusted within their pool, so this cross-network initialization authority is accepted by design and retained only as an Informational boundary note.

Recommended Mitigation: In `NAVManager.fromSpoke()`, require `assetId.centrifugeId() == centrifugeId` for both initialization actions. For ISO assets whose encoded network is zero, store and enforce an explicit authoritative network rather than treating zero as unrestricted. Consider further binding manager grants to the permitted share class, asset, and holding/liability role. The existing cross-network test does not cover this invariant because it changes the authenticated source network, causing the manager check itself to fail, instead of keeping the

authorized source fixed and supplying a foreign-network asset. Add regression tests where a network-A manager submits each initialization action with a network-B asset and expect a source-mismatch revert, alongside positive matching-network tests.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: The fix sufficiently resolves the issue.

7.3.3 Failed setPoolAdapters Retry Bypasses Session Revocation

Target: `src/core/messaging/Gateway.sol` (`failedMessages`, `_safeProcess`, and `retry`), `src/core/messaging/MultiAdapter.sol` (`setAdapters`, `blockSession`, `_execute`, and `_resolve`), and `src/core/messaging/MessageProcessor.sol` (`_handleSpoke`)

Severity: Informational

Description: An attacker controlling enough adapters to satisfy active session N 's threshold—which on deployed threshold-one routes means a single adapter—can authenticate a `SetPoolAdapters` message targeting session $N + 2$ and specifying an attacker-controlled threshold-one adapter set. After the message reaches quorum, `MultiAdapter` removes its two-byte session prefix and forwards only the body to `Gateway`. Because `setAdapters` currently expects $N + 1$, processing reverts; however, `Gateway._safeProcess` converts the rejection into a durable, non-expiring failed-message credit keyed only by the source chain and body hash, without retaining the authenticated session.

If governance subsequently installs the legitimate session $N + 1$ and calls `blockSession` for session N , the credit remains because blocking only clears `MultiAdapter`'s session configuration. Anyone can then call permissionless `Gateway.retry`, which sends the body directly to `MessageProcessor` without rechecking its original session or quorum. The same update now satisfies the $N + 2$ freshness check and installs the attacker's adapter set despite the revocation of session N . The attacker thereby regains cross-chain authentication authority for the affected pool and can approve arbitrary pool-scoped messages, including further adapter rotations, manager updates, manager calls, and share transfers. An authorized `clearFailedMessage` transaction is not a reliable safeguard because a public retry can execute first.

Recommended Mitigation: Preserve the authenticated session across the `MultiAdapter`-to-`Gateway` boundary and key each failed credit by the source chain, message hash, originating session, and that session's revocation generation. Increment the affected session's generation in `blockSession`, and allow `retry` to consume only a credit whose recorded generation still matches and whose originating session remains live. Add a regression test in which a session- N update targeting $N + 2$ fails, governance installs $N + 1$ and blocks N , and the queued update can no longer be retried.

Centrifuge: The attacker already needs quorum-level control, which provides equivalent control over the affected pool-to-spoke route; therefore, the issue is Informational.

BurraSec: Acknowledged, we confirm that the bug adds persistence but no incremental authority beyond the prerequisite quorum compromise.

7.3.4 `IHoldings.decrease` documents the wrong valuation behavior

Target: `IHoldings.sol`, `Holdings.sol`

Severity: Informational

Description: `IHoldings.decrease` states that a realized decrease is quoted through the current hub-side valuation and may be blocked by a stale or unset oracle. The implementation deliberately performs no valuation call: it removes the realized fraction of the holding's already stored carrying value.

Recommended Mitigation: Update the interface to state that decreases remove stored carrying value pro rata and do not query the current valuation.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: Fix looks good

7.3.5 `Holdings.increase` unnecessarily quotes a zero realized value

Target: `Holdings.sol`

Severity: Informational

Description: When an increase only repays a preceding over-decrease, realized is zero. `Holdings.increase` still calls `getQuote(..., 0)` and executes a same-value carrying-value update. Canonical valuations return zero but first perform unnecessary registry reads, conversion work and, for `OracleValuation`, price loading and validation.

Recommended Mitigation: When `realized == 0`, preserve the valuation-validity check but skip quote conversion and the carrying-value write. Alternatively, define zero-quote and validity semantics in `IValuation` before bypassing valuation entirely.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: Fix looks good

7.3.6 `ITokenBridge` documents the wrong refund and return behavior

Target: `ITokenBridge.sol`, `TokenBridge.sol`

Severity: Informational

Description: `ITokenBridge.send` documents its return value as the response from the token's handler. However, `TokenBridge.send` never obtains such a response and unconditionally returns `bytes("")`. Integrators therefore cannot rely on the documented return value.

Recommended Mitigation: Document that it is retained for compatibility and always returns empty bytes.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#). Only fixed the return bytes comment. For the `refundAddress`, even if the intermediate address is the `relayer`, eventually the `refundAddress` should receive any excess.

BurraSec: Fix looks good

7.3.7 Cross-chain documentation names the wrong execution components

Target: `ITransferHook.sol`, `ISpoke.sol`, `MessageLib.sol`

Severity: Informational

Description: `ITransferHook` still identifies `crosschainSource` as `Spoke`, but the canonical deployment configures `SpokeHandler`, and `SpokeHandler.executeTransferShares` performs inbound share execution. Separately, `ISpoke` and `MessageLib` describe the bridging hook that consumes the forwarded sender as destination-side, although `HubHandler.initiateTransferShares` invokes that hook on the Hub before dispatching the transfer to the destination Spoke. These stale component names obscure which contract must be configured as the cross-chain execution source and where bridge policy is evaluated.

Recommended Mitigation: Identify `SpokeHandler` as the `crosschainSource` and inbound execution contract, and describe the bridging hook as Hub-side.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: Fix looks good

7.3.8 Spoke performs a redundant share-class registry lookup

Target: Spoke.sol, SpokeRegistry.sol

Severity: Informational

Description: Every `Spoke.crosschainTransferShares` call first invokes `hasShareClass(poolId, scId)` and then invokes `shareTokenAndRegistrar(poolId, scId)`, causing two external reads of the same registry entry. The second call already returns the token needed to establish existence.

Recommended Mitigation: Remove the separate `hasShareClass` call, cache the token and registrar returned by `shareTokenAndRegistrar`, and reject a zero token address. For example:

```
(IERC20 share, IRegistrar registrar) = spokeRegistry.shareTokenAndRegistrar(poolId, scId);
require(address(share) != address(0), ISpokeRegistry.ShareTokenDoesNotExist());

require(centrifugeId != sender.localCentrifugeId(), LocalTransferNotAllowed());
require(registrar.canBridge(address(share), owner, centrifugeId, amount), BridgeNotAllowed());
```

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: Fix looks good

7.3.9 VaultRouter queries vault.share() twice

Target: VaultRouter.sol

Severity: Informational

Description: `VaultRouter.crosschainTransferShares` calls the external `vault.share()` getter once when pulling shares and again when approving Spoke. Since the share address is the same between those statements in the canonical vault, the second external call is unnecessary.

Recommended Mitigation: Cache `vault.share()` in a local variable and reuse it for both the transfer and approval.

Centrifuge: Fixed in [centrifuge/protocol-internal#307](#)

BurraSec: Fix looks good